Formalization of a Proof Calculus for Incremental Linearization for NTA Satisfiability

Tomaz Mascarenhas[1], Harun Khan[1], Abdalrhman Mohamed, Andrew Reynolds, Haniel Barbosa, Clark Barrett, Cesare Tinelli

SC^2, Stuttgart, 02/08/2025

[1]Equal contribution

Background

Nonlinear Arithmetic and Transcendental Functions

- The NTA theory in SMT comprises nonlinear arithmetic and some transcendental functions
  - Trigonometric functions, $\exp$ and $\log$

Nonlinear Arithmetic and Transcendental Functions

- The NTA theory in SMT comprises nonlinear arithmetic and some transcendental functions
  - Trigonometric functions, $\exp$ and $\log$
- Applications in formal verification of dynamic systems, digital signal processing and motion planning for robots
  - Automation in proof assistants, if provided the necessary tooling

Nonlinear Arithmetic and Transcendental Functions

- The NTA theory in SMT comprises nonlinear arithmetic and some transcendental functions
  - Trigonometric functions, $\exp$ and $\log$
- Applications in formal verification of dynamic systems, digital signal processing and motion planning for robots
  - Automation in proof assistants, if provided the necessary tooling
- QF_NRA requires exponential time to solve and QF_NTA has been shown to be to be undecidable

Incremental Linearization

- *Incremental linearization* is a method introduced by Cimatti et al. (2018) for deciding the satisfiability of formulas in QF_NTA
  - Not a decision procedure

Incremental Linearization

- *Incremental linearization* is a method introduced by Cimatti et al. (2018) for deciding the satisfiability of formulas in QF_NTA
  - Not a decision procedure

- Given a formula ψ in QF_NTA, a new formula ψ' is obtained by abstracting each transcendental function and variable multiplication as an uninterpreted function

Incremental Linearization

- *Incremental linearization* is a method introduced by Cimatti et al. (2018) for deciding the satisfiability of formulas in QF_NTA
  - Not a decision procedure

- Given a formula ψ in QF_NTA, a new formula ψ' is obtained by abstracting each transcendental function and variable multiplication as an uninterpreted function

- ψ' is thus a formula in QF_UFLRA. If ψ' is unsatisfiable, then so is ψ

Incremental Linearization

- If ψ' is satisfiable, not necessarily ψ is. In this case, the method incrementally introduces new lemmas blocking spurious solutions

Incremental Linearization

- If ψ' is satisfiable, not necessarily ψ is. In this case, the method incrementally introduces new lemmas blocking spurious solutions

Example

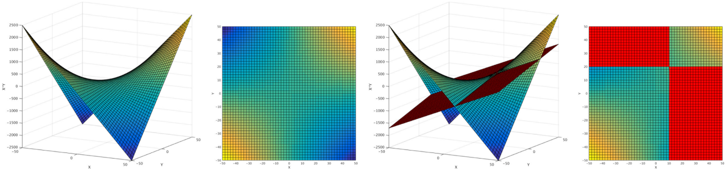
Let $\psi = x^2 + y^2 \leq 2 \land (x \leq -1.1 \lor x \geq 1.1) \land (y \leq -1.1 \lor y \geq 1.1)$. ψ' is the same, but instead of x^2 and y^2 we have $f_*(x, x)$ and $f_*(y, y)$. It is possible to interpret f_* in such a way that ψ' is satisfied, but the original formula is unsatisfiable.

Incremental Linearization

- Spurious solutions involving variable multiplication are blocked through the *tangent plane* equation and some sign properties

Incremental Linearization

- Spurious solutions involving variable multiplication are blocked through the *tangent plane* equation and some sign properties

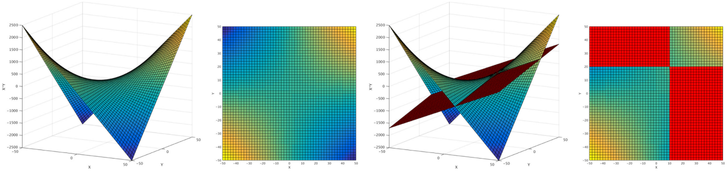


(a) $x * y$ (b) $x * y$ (top view) (c) $x * y$ and tangent plane (d) $x * y$ and tangent plane (top view)

Source: Cimatti et al.

Incremental Linearization

▷ Spurious solutions involving variable multiplication are blocked through the *tangent plane* equation and some sign properties



(a) $x * y$ (b) $x * y$ (top view) (c) $x * y$ and tangent plane (d) $x * y$ and tangent plane (top view)

Source: Cimatti et al.

▷ Spurious solutions involving transcendental functions are blocked using their *Taylor polynomial*

Incremental Linearization

function $\textsc{IncrementalLinearization}(\psi)$
    $\psi' \leftarrow \textsc{InitialAbstraction}(\psi)$
    $\Gamma \leftarrow \varnothing$
    while $\textsc{InTimeLimit}()$ **do**
        $\langle sat, \mu \rangle \leftarrow \textsc{SolveUFLRA}(\psi' \wedge \Gamma)$
        if not sat **then return** UNSAT
        $\langle sat', \Gamma' \rangle \leftarrow \textsc{CheckRefine}(\psi', \mu)$
        if sat' **then return** SAT
        $\Gamma \leftarrow \Gamma \cup \Gamma'$

Proof Calculus

- cvc5 implements a slightly modified version of this method

Proof Calculus

- cvc5 implements a slightly modified version of this method
- The solver also defines a *proof calculus* composed of 21 lemmas capturing its reasoning

Proof Calculus

- cvc5 implements a slightly modified version of this method
- The solver also defines a *proof calculus* composed of 21 lemmas capturing its reasoning
- Types of lemmas
  - Rules eliminating spurious solutions
  - Phase shifting for $\sin$
  - Bounding π

Other transcendental functions

- $\sin$ and $\exp$ are the only functions directly handled

Other transcendental functions

- $\sin$ and $\exp$ are the only functions directly handled
- $\cos(x) = \sin(x + \frac{\pi}{2})$

Other transcendental functions

- $\sin$ and $\exp$ are the only functions directly handled
- $\cos(x) = \sin(x + \frac{\pi}{2})$
- $\tan(x) = \frac{\sin(x)}{\cos(x)}$

Other transcendental functions

- $\sin$ and $\exp$ are the only functions directly handled
- $\cos(x) = \sin(x + \frac{\pi}{2})$
- $\tan(x) = \frac{\sin(x)}{\cos(x)}$
- $\sec(x) = \frac{1}{\cos(x)}$

Other transcendental functions

- $\sin$ and $\exp$ are the only functions directly handled
- $\cos(x) = \sin(x + \frac{\pi}{2})$
- $\tan(x) = \frac{\sin(x)}{\cos(x)}$
- $\sec(x) = \frac{1}{\cos(x)}$
- $\log$ and inverse trigonometric functions are modeled with new variables. For instance, if the input formula has the term $\log(1 + x)$ we introduce a new variable y and add the assertion $\exp(y) = 1 + x$

Contributions

- In this work we mechanize proofs of all the lemmas in the proof calculus using the Lean proof assistant
  - *lean-smt*

Contributions

- In this work we mechanize proofs of all the lemmas in the proof calculus using the Lean proof assistant
  - *lean-smt*
- In this process, we identified some errors in the documentation, which were fixed

Contributions

- ▷ In this work we mechanize proofs of all the lemmas in the proof calculus using the Lean proof assistant
  - *lean-smt*
- ▷ In this process, we identified some errors in the documentation, which were fixed
- ▷ Increase confidence in the proof calculus and, once cvc5 emits proofs using the calculus, in its implementation

Formalization of the Proof Calculus

Lemmas on multiplication - ARITH_MULT_TANGENT

$$\frac{- \mid x, y, a, b}{xy \leq bx + ay - ab \leftrightarrow ((x \leq a \land y \geq b) \lor (x \geq a \land y \leq b))}$$

$$\frac{- \mid x, y, a, b}{xy \geq bx + ay - ab \leftrightarrow ((x \leq a \land y \leq b) \lor (x \geq a \land y \geq b))}$$

Lemmas on multiplication - ARITH_MULT_TANGENT

```
theorem arithMulTangentLower (x y a b : ℝ) :
    x * y ≤ b * x + a * y - a * b ↔ ((x ≤ a ∧ y ≥ b) ∨ (x ≥ a ∧ y ≤ b))
```

```
theorem arithMulTangentUpper (x y a b : ℝ) :
    x * y ≥ b * x + a * y - a * b ↔ ((x ≤ a ∧ y ≤ b) ∨ (x ≥ a ∧ y ≥ b))
```

Lemmas on multiplication - ARITH_MULT_SIGN

$$\frac{f_1 \wedge \cdots \wedge f_k \mid m}{m \bowtie 0}$$

Each f_i has the form $x_j \bowtie 0$. m is a monomial composed by powers of the x_i's. The relation in the conclusion is defined based on the parity of the exponents and on the sign of each x_i

Lemmas on multiplication - ARITH_MULT_SIGN

```
theorem powNegOdd       : ∀ {k : ℕ} {r : ℝ}, r < 0 → Odd k  → r ^ k < 0
theorem powNegEven      : ∀ {k : ℕ} {r : ℝ}, r < 0 → Even k → r ^ k > 0
theorem nonZeroEvenPow  : ∀ {k : ℕ} {r : ℝ}, r ≠ 0 → Even k → r ^ k > 0
theorem powPos          : ∀ {k : ℕ} {r : ℝ}, r > 0          → r ^ k > 0
```

```
theorem combineSigns₁ : ∀ {a b : ℝ}, a > 0 → b > 0 → b * a > 0
theorem combineSigns₂ : ∀ {a b : ℝ}, a > 0 → b < 0 → b * a < 0
theorem combineSigns₃ : ∀ {a b : ℝ}, a < 0 → b > 0 → b * a < 0
theorem combineSigns₄ : ∀ {a b : ℝ}, a < 0 → b < 0 → b * a > 0
```

Lemmas for transcendental functions - Bounding $\exp$ and $\sin$

$$\frac{d \text{ odd} \mid d, c, t}{t \geq c \to \exp(t) \geq \texttt{maclaurin}(\exp, d, c)}$$

$$\frac{d \equiv 3 \text{ mod } 4^* \wedge (0 < l \wedge u \leq \pi)^* \mid d, t, l, u}{t \in [l, u] \to \sin(t) \geq \texttt{secant}(\sin, l, u, t)}$$

Where `secant` is the secant line to the d-th Taylor polynomial at the points $(l, p(l))$ and $(u, p(u))$

Taylor's theorem and Lagrange's remainder

- These results follow from *Taylor's theorem* with the Lagrange form of the remainder:

$$\exists x' \in [x_0, x], f(x) - \left(\sum_{j=0}^{n} \frac{f^{(j)}(x_0)}{j!} (x - x_0)^j \right) = \frac{f^{(n+1)}(x')}{(n+1)!} (x - x_0)^{n+1}$$

Taylor's theorem and Lagrange's remainder

- These results follow from *Taylor's theorem* with the Lagrange form of the remainder:

$$\exists x' \in [x_0, x], f(x) - \left(\sum_{j=0}^{n} \frac{f^{(j)}(x_0)}{j!}(x - x_0)^j \right) = \frac{f^{(n+1)}(x')}{(n+1)!}(x - x_0)^{n+1}$$

- This formula already existed in mathlib, but with the additional hypothesis that $x_0 < x$. We extended it to the general version

Lemmas for transcendental functions - Bounding $\exp$ and $\sin$

```
theorem arithTransExpApproxBelow (d c t : ℕ) :
  Odd d → t ≥ c → Real.exp t ≥ taylorWithinEval Real.exp d Set.univ 0 c
```

```
theorem arithTransSineApproxBelowPos (d k : ℕ) (hd : d = 4 * k + 3) (t l u : ℝ)
    (ht : l ≤ t ∧ t ≤ u)
    (hl : 0 < l) (hu : u ≤ Real.pi) :
    let p : ℝ → ℝ := taylorWithinEval Real.sin d Set.univ 0
    sin t ≥ ((p l - p u) / (l - u)) * (t - l) + p l
```

Future Work

- Proof reconstruction using *lean-smt*
  - cvc5 emits a proof written in terms of these rules and the rest of its proof calculus (CPC)
  - lean-smt translates this to a Lean proof and checks it (the Lean theorem is inferred)
  - Or one can start with a Lean goal and use lean-smt to translate it to an SMT-LIB query

Future Work

- Proof reconstruction using *lean-smt*
  - cvc5 emits a proof written in terms of these rules and the rest of its proof calculus (CPC)
  - lean-smt translates this to a Lean proof and checks it (the Lean theorem is inferred)
  - Or one can start with a Lean goal and use lean-smt to translate it to an SMT-LIB query

- Challenges with real numbers

Future Work

- Proof reconstruction using *lean-smt*
  - cvc5 emits a proof written in terms of these rules and the rest of its proof calculus (CPC)
  - lean-smt translates this to a Lean proof and checks it (the Lean theorem is inferred)
  - Or one can start with a Lean goal and use lean-smt to translate it to an SMT-LIB query
- Challenges with real numbers
- Arbitrary π approximation

Thanks! Questions?